Documento técnico final

Sistema de Trazabilidad e Inventarios UADY: integración distribuida con el patrón Broker.

1. Introducción

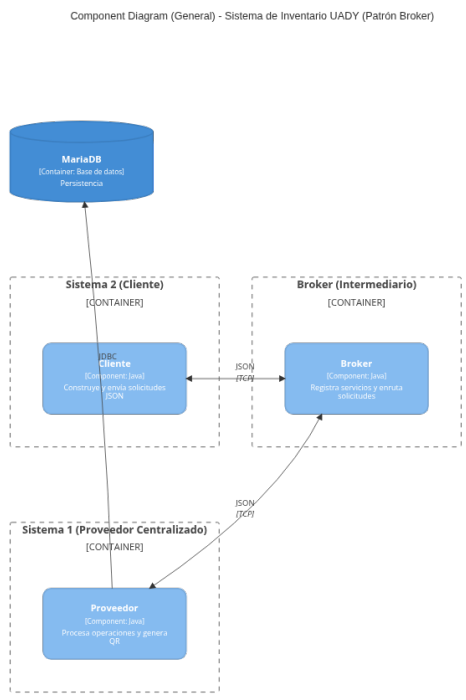
El codigo del proyecto se puede encontrar en este repositorio.

El proyecto integra dos sistemas que gestionan el inventario de los almacenes de la UADY:

- **Sistema 1 (centralizado / proveedor):** registra los productos y cada unidad física (mercancía), genera un código QR por unidad y registra las ventas. Incluye una interfaz Swing para el Administrador.
- **Sistema 2 (cliente del proveedor):** aplicación de escritorio con la que un proveedor registra a qué tienda de la UADY se vendió una unidad y consulta su código QR.

Ambos sistemas se comunican a través de un **Broker**, un tercer proceso independiente que registra los servicios disponibles y enruta cada solicitud al proveedor correspondiente. La comunicación usa Sockets TCP con mensajes JSON (marshalling y unmarshalling con Gson), atención concurrente con hilos, registro dinámico de servicios, timeouts y manejo de desconexiones.

2. Arquitectura



Componente	Responsabilidad
Cliente (Sistema 2)	

Componente	Responsabilidad
	Construye solicitudes JSON, las envía al Broker y presenta la respuesta. Solo conoce la dirección del Broker.
Broker	Registra proveedores, identifica el servicio pedido, reenvía la solicitud y devuelve la respuesta.
Proveedor (Sistema 1)	Ejecuta la lógica de negocio, genera los QR y persiste en MariaDB. Se registra en el Broker al arrancar.
MariaDB	Almacena productos, mercancía, tiendas, usuarios, ventas e historial de precios.

Tecnologías: Java 21, Maven multimódulo, Gson, Java Swing con FlatLaf, ZXing (QR), MariaDB con JDBC (MariaDB Connector/J) y Docker Compose.

3. Contrato de comunicación

Cada mensaje es un objeto JSON en una sola línea. El módulo `common` define el contrato que comparten los tres componentes:

```
{"idSolicitud": "ev-04", "nombreServicio": "gestorTiendas", "nombreOperacion":
"actualizarTiendaVenta",
 "parametros": {"numero_serie": "SN-UADY-0003", "id_tienda": 1, "id_usuario":
4}, "fechaHora": "..."}

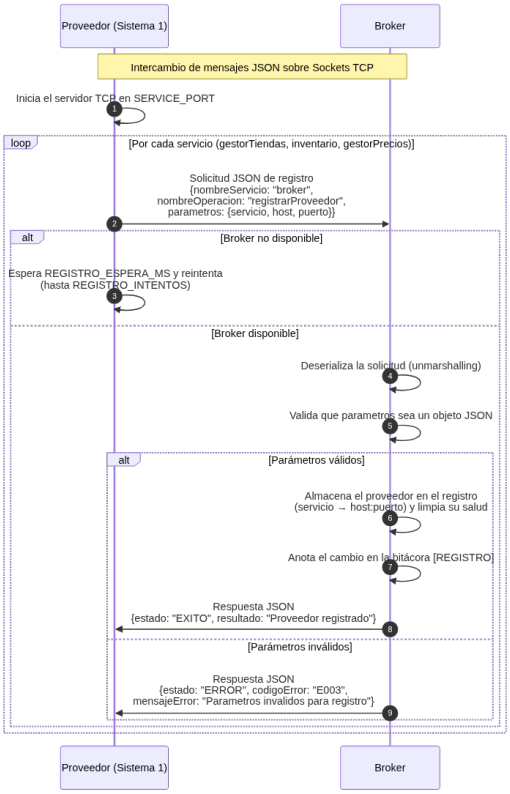
{"idSolicitud": "ev-04", "estado": "EXITO",
 "resultado": {"numero_serie": "SN-UADY-0003", "id_tienda": 1, "fecha_venta":
"..."}, "fechaHora": "..."}
```

Servicio	Operaciones	Atendido por
`broker`	`registrarProveedor`, `actualizarProveedor`, `removerProveedor`	Broker
`gestorTiendas`	`listarTiendas`, `listarUsuarios`, `actualizarTiendaVenta`	Sistema 1
`inventario`	`listarInventario`, `obtenerQr`	Sistema 1
`gestorPrecios`	`listarPreciosProducto`	Sistema 1

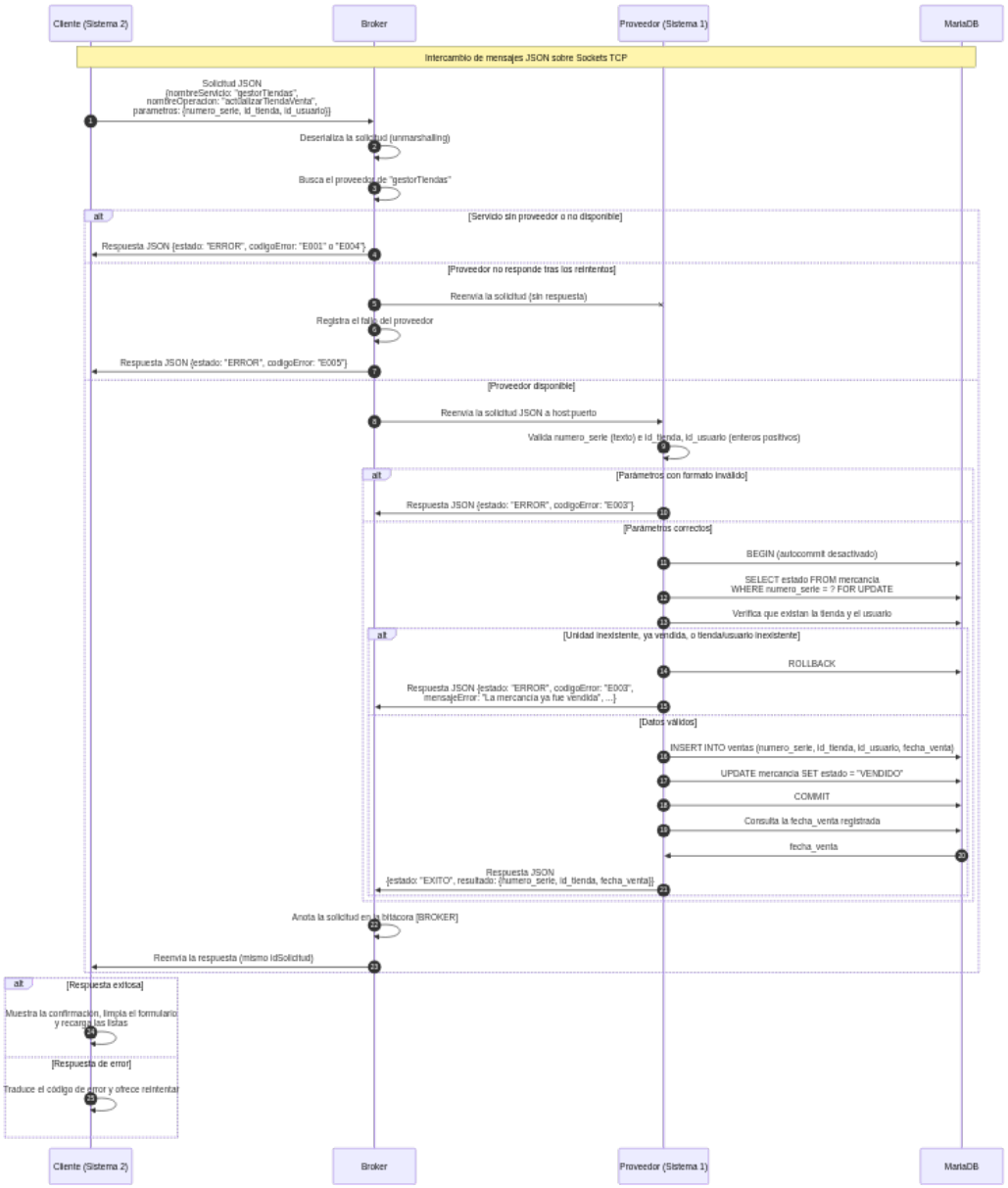
Los errores se responden con `estado: "ERROR"`, un código y un mensaje legible: `SERVICIO_DESCONOCIDO` (E001), `OPERACION_NO_SOPORTADA` (E002), `PARAMETROS_INVALIDOS` (E003), `PROVEEDOR_NO_REGISTRADO` (E004), `TIMEOUT` (E005) y `ERROR_INTERNO` (E999). La respuesta siempre conserva el `idSolicitud` de la solicitud.

4. Flujos principales

Registro dinámico. Al arrancar, el Sistema 1 registra cada uno de sus servicios en el Broker y reintenta si el Broker todavía no está disponible:



Registro de una venta. Es la operación central del sistema. El Broker localiza al proveedor y el Sistema 1 registra la venta en una transacción con bloqueo pesimista (`SELECT ... FOR UPDATE`), de modo que una unidad no pueda venderse dos veces:



Las demás operaciones (listados de tiendas, usuarios, inventario e historial de precios, obtención del QR y administración de proveedores) siguen el mismo patrón de enrutamiento. Sus diagramas están en el documento Diagramas de secuencia.

5. Diseño interno

Módulo	Organización
`common`	`contrato` (Solicitud, Respuesta), `error`, `json` (GsonFactory), `modelo` y `servicio` (catálogo de servicios y operaciones).
`broker`	`network` (servidor y un manejador por conexión), `registry` (registro y salud de proveedores), `routing` (enrutador con reintentos) y `util` (configuración y bitácora).

Módulo	Organización
`sistema-centralizado`	Capas Modelo-Servicio-Repositorio: `network` (servidor TCP y despachador), `service` (negocio y QR), `repository` (JDBC); interfaz MVC con `view` y `controller`.
`sistema-cliente`	MVC: `view` (Swing), `controller` (tareas en segundo plano con `SwingWorker`) y `model` (cliente TCP y DTOs).

El Sistema 1 abre su interfaz solo cuando hay pantalla; en Docker arranca únicamente como servicio. La interfaz y el servidor de red comparten los mismos servicios sobre MariaDB.

6. Concurrencia y resiliencia

- **Hilos:** el Broker y el Sistema 1 atienden cada conexión en un hilo de un pool (16 hilos por defecto).
- **Timeouts separados:** conexión al proveedor (2000 ms) y lectura de la respuesta (5000 ms).
- **Reintentos limitados:** 2 reintentos adicionales por solicitud ante fallos de red, con espera entre intentos. Una respuesta de error válida del proveedor no se reintenta.
- **Proveedor no disponible:** tras 3 solicitudes fallidas consecutivas, el servicio se marca como no disponible durante 30 s y se rechaza de inmediato; un nuevo registro lo reactiva.
- **Bitácora:** cada solicitud se anota con su id, operación, proveedor destino, resultado y tiempo (`[BROKER]`), junto con los cambios del registro (`[REGISTRO]`).

Todos los valores se configuran con variables de entorno (`BROKER_TIMEOUT_*`, `BROKER_REINTENTOS`, `BROKER_FALLOS_PARA_INDISPONIBLE`, `BROKER_INDISPONIBLE_MS`).

7. Despliegue

El sistema se levanta con `docker compose up --build`, después de copiar `.env.example` a `.env` y definir las contraseñas de la base de datos. Compose respeta el orden de arranque mediante health checks: MariaDB, luego el Broker y por último el Sistema 1, que se registra en el Broker.

Componente	Puerto	Notas
MariaDB	3306	Se inicializa con `01_schema.sql` y `02_seed.sql`.
Broker	5000	Registro de servicios en memoria.
Sistema 1	6000	Los QR se guardan en el volumen `qr_storage`.
Sistema 2	—	Se ejecuta desde el IDE o como jar en Windows, o en Docker con el perfil `client` y un servidor X11.

8. Validación

Las pruebas se ejecutaron sobre el stack de Docker Compose con clientes que solo conocen la dirección del Broker. Cada evidencia incluye el JSON enviado, el recibido y la bitácora del Broker.

#	Prueba	Resultado
1	Registro de proveedores	Los 3 servicios quedan registrados hacia `sistema-centralizado:6000`.
2	Localización de un servicio	El Broker localiza al proveedor de `inventario` y responde `EXITO`.
3	Ejecución por servicio	`listarTiendas`, `listarInventario` y `listarPreciosProducto` responden `EXITO`.
4	Respuesta exitosa	La venta devuelve id, resultado, estado y fecha; la unidad queda `VENDIDO`.
5	Servicio desconocido	`SERVICIO_DESCONOCIDO`, sin contactar a ningún proveedor.
6	Parámetro inválido	`PARAMETROS_INVALIDOS` con un mensaje claro.
7	Excepción del proveedor	`ERROR_INTERNO`; el Broker sigue atendiendo y se recupera sin reiniciar.
8	Cambio de proveedor	El mismo cliente lo atiende un segundo proveedor sin modificarse.
9	Proveedor caído	3 intentos por solicitud y, tras 3 fallos, servicio `NO DISPONIBLE`.
10	Pruebas automatizadas	`mvn test`: **195 pruebas, 0 fallos** (Common 10, Broker 47, Sistema 1 129, Sistema 2 9).
11	Concurrencia	20 clientes simultáneos: 20/20 reciben su propia respuesta, atendidos por 16 hilos.
12	Broker caído	Sin Broker no hay comunicación; al reiniciarlo, su registro está vacío.
13	Solicitud duplicada	La segunda venta se rechaza y en la base de datos queda una sola.
14	Costo en rendimiento	El Broker agrega 0.92 ms por solicitud (3.53 ms directo frente a 4.45 ms).

9. Preguntas de reflexión

Las respuestas completas, con fragmentos de código y bitácoras, están en el documento Preguntas de la actividad; los números de prueba remiten a la sección 8.

1. ¿Cómo reduce el Broker el acoplamiento? El cliente no conoce ni instancia al proveedor: solo conoce la dirección del Broker y envía nombres de servicio y operación. El Broker localiza al proveedor en su registro y enruta la solicitud, de modo que el proveedor se sustituyó sin modificar el cliente (prueba 8).

2. ¿Qué detalles oculta al cliente? La ubicación de red del proveedor, su implementación, la base de datos, la generación de QR y la ruta interna de los PNG. El cliente solo ve el contrato JSON; la dirección `sistema-centralizado:6000` aparece únicamente en la bitácora del Broker (prueba 2).

3. ¿Qué sucede si el Broker deja de funcionar? Se interrumpe la integración: el cliente recibe un error de conexión aunque el Sistema 1 siga activo. Como el registro vive en memoria, al reiniciar el Broker los servicios aparecen como desconocidos hasta que el proveedor se vuelve a registrar (prueba 12).

4. ¿El Broker constituye un punto único de falla? Sí. El cliente tiene configurada una sola dirección de Broker y no existe otra ruta hacia el proveedor; con el Broker detenido la solicitud falla aunque MariaDB y el Sistema 1 estén en ejecución.

5. ¿Cómo se podría mejorar su disponibilidad? Con réplicas del Broker, failover, balanceo de carga, health checks, reconexión automática y registro redundante. El proyecto ya implementa timeouts separados de conexión y lectura, reintentos limitados y la marca de proveedor no disponible (prueba 9); las réplicas, el balanceo y el re-registro periódico quedan como mejoras.

6. ¿Qué atributo mejora al registrar servicios dinámicamente? La flexibilidad y, con ella, la mantenibilidad y la escalabilidad: los proveedores se agregan o sustituyen en caliente con `broker.registrarProveedor`, sin detener el Broker ni modificar el cliente (pruebas 1 y 8).

7. ¿Qué costo introduce el Broker en rendimiento? Un salto de red adicional, serialización y deserialización extra, enrutamiento y el costo de sockets e hilos. Medido sobre 100 solicitudes, agrega 0.92 ms en promedio (26 %) frente a la llamada directa (prueba 14). Mal dimensionado, puede volverse un cuello de botella.

8. ¿Dónde se realiza el marshalling y unmarshalling? En los tres componentes, con la misma `GsonFactory` de `common`: el cliente serializa al enviar y deserializa al recibir; el Broker deserializa la solicitud, la vuelve a serializar hacia el proveedor y repite el proceso con la respuesta; el proveedor hace lo propio (prueba 5).

9. ¿Cómo se gestionarían solicitudes duplicadas? Con una idempotency key: cada solicitud ya lleva un `idSolicitud` único (UUID) que permitiría detectar repetidos y devolver la respuesta original. Hoy la duplicación de una venta la evita la transacción con `SELECT ... FOR UPDATE`: la segunda solicitud encuentra la unidad `VENDIDO` y se rechaza (prueba 13).

10. ¿Cómo se monitorearía el Broker? Con bitácora por id de solicitud, métricas de latencia, throughput, conexiones y timeouts, health checks, alertas y monitoreo de CPU, memoria e hilos. El proyecto ya registra cada solicitud con su id, hilo, proveedor, resultado y latencia, y el Broker tiene un health check en Docker Compose (prueba 11).

10. Limitaciones conocidas

- **Registro volátil:** el registro del Broker vive en memoria y el Sistema 1 solo se registra al arrancar. Tras reiniciar el Broker, los servicios aparecen como desconocidos hasta reiniciar el proveedor.
- **Timeouts desalineados:** cuando el proveedor no responde, el Broker tarda unos 15 s (3 intentos de 5 s) en contestar, pero el Sistema 2 espera solo 5 s, así que el usuario ve antes su propio timeout.
- **Zona horaria:** los contenedores usan UTC, por lo que la bitácora y `fecha_venta` muestran 6 horas más que la hora local.
- **Respuestas de administración:** `actualizarProveedor` y `removerProveedor` responden éxito aunque el servicio no estuviera registrado; solo la bitácora indica si el cambio se aplicó.

11. Conclusiones

1. **El patrón Broker cumplió su objetivo de desacoplar.** El cliente no conoce la ubicación ni la implementación del proveedor, y se pudo sustituir el proveedor en ejecución sin modificar ni reiniciar el cliente. El contrato común en `common` fue clave: los tres componentes comparten las mismas clases de solicitud, respuesta y códigos de error.
2. **El desacoplamiento tiene un costo bajo pero real.** El salto adicional agrega menos de un milisegundo por solicitud, aceptable para este dominio, a cambio de flexibilidad y de un punto central para controlar errores y bitácora.
3. **El Broker concentra el riesgo.** Es un punto único de falla y su registro en memoria se pierde al reiniciarlo. Las mejoras naturales son el re-registro periódico de los proveedores, la persistencia del registro y la replicación del Broker.
4. **La resiliencia debe diseñarse de extremo a extremo.** Los reintentos y la marca de proveedor no disponible protegen al sistema, pero los timeouts del cliente y del Broker deben alinearse para que el usuario reciba el error correcto.
5. **La consistencia se resolvió en la capa de datos.** La transacción con bloqueo pesimista evita ventas duplicadas y condiciones de carrera aun con clientes concurrentes; una clave de idempotencia complementaría esta protección a nivel de mensajes.
6. **El proyecto quedó verificado y reproducible.** Las 195 pruebas automatizadas, las 14 evidencias de ejecución y el despliegue con Docker Compose permiten reproducir el comportamiento del sistema en cualquier equipo.